

UNIDAD DE PROGRAMACIÓN II.

Utilización de modelos de IA.

Parte I. Requisitos básicos de un sistema de resolución de problemas.

Autor: Pedro Antonio Santiago Santiago.
2025



CC BY-NC-SA 4.0

Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, for noncommercial purposes only. If others modify or adapt the material, they must license the modified material under identical terms.

👤 **BY:** Credit must be given to you, the creator.

💰 **NC:**
Only noncommercial use of your work is permitted. *Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.*

🔄 **SA:** Adaptations must be shared under the same terms.

[See the License Deed](#)

1. Ficha pedagógica.

Temporalización: 18 horas.	
RESULTADOS DE APRENDIZAJE	RA2
OBJETIVOS GENERALES	b, c, d,, j, k, m,o, p
COMPETENCIAS	b, c, d, e,k, n,ñ, p, r
CONTENIDOS	
Requisitos básicos de un sistema de resolución de problemas. Modelos de sistemas de Inteligencia Artificial: Automatización de tareas. Sistemas de razonamiento impreciso. Sistemas basados en reglas.	
ORIENTACIONES METODOLÓGICAS	
Relacionar las actividades con el entorno socio-económico para despertar el interés por el módulo. Se realizan actividades de introducción que permiten a los alumnos descubrir la necesidad de los contenidos tratados en el mundo laboral. Las actividades se dirigen a potenciar la documentación de trabajos realizados y lectura de documentación técnica.	
CRITERIO DE EVALUACIÓN	a) Se han determinado los requisitos básicos a implementar en un sistema de resolución de problemas. b) Se han clasificado modelos de Inteligencia Artificial. f) Se ha valorado la adecuación de los modelos a la implementación del sistema de resolución de problemas.

2. Requisitos básicos de un sistema de resolución de problemas.

Una de las principales finalidades de la computación es la resolución de problemas, se han diseñado diferentes técnicas para resolver estos. Este tema se centra en la resolución de los problemas algorítmicos y de optimización tratando las técnicas más utilizadas, ya que tratarlos todos no es viable temporalmente.

Algunas de las técnicas más utilizadas:

Técnica de resolución	Descripción básica	Problemas clásicos
-----------------------	--------------------	--------------------

Recursividad básica	Un problema se resuelve definiéndose en términos de sí mismo, hasta llegar a un caso base. Suele ser la base de otras técnicas más avanzadas.	- Factorial.- Serie de Fibonacci (versión ingenua). - Torres de Hanói. - Recorridos en árboles (preorden, inorden, postorden).
Divide y vencerás	Divide el problema en subproblemas más pequeños, los resuelve y combina resultados.	- MergeSort (ordenamiento). - QuickSort (ordenamiento). - Búsqueda binaria.
Programación dinámica (DP)	Resuelve problemas con subproblemas superpuestos, guardando resultados para no recalcular.	- Fibonacci optimizado. - Mochila 0/1. - Edición de cadenas. - Multiplicación de matrices.
Backtracking	Explora todas las soluciones posibles de forma recursiva, retrocediendo cuando no hay solución válida.	- N-Reinas. - Sudoku. - Laberintos. - Coloreo de grafos.
Algoritmos voraces (Greedy)	Construyen la solución paso a paso, eligiendo siempre la mejor opción local.	- Kruskal (árbol de expansión mínima). - Prim. - Dijkstra. - Cambio de monedas.
Búsqueda en grafos (BFS / DFS)	Recorren sistemáticamente grafos o espacios de estados. BFS asegura la ruta más corta en aristas uniformes; DFS explora profundo.	- BFS: rutas mínimas sin peso. - DFS: detección de ciclos, laberintos. - Árboles de decisión.
Búsqueda heurística (A*)	Usa heurísticas para guiar la búsqueda hacia soluciones óptimas con mayor eficiencia.	- Ruta más corta en mapas. - 8-puzzle. - Planeamiento de rutas.
Programación lineal / entera	Formula el problema como funciones y restricciones lineales, resuelto con métodos exactos.	- Transporte. - Asignación de recursos. - Planificación.

Metaheurísticas	Métodos aproximados para problemas complejos, buscando buenas soluciones en tiempo razonable.	- Algoritmos genéticos. - Recocido simulado. - Colonia de hormigas.
------------------------	---	---

Este tema se centra en: repasar la recursividad básica, sobre la que se sustentan el resto de técnicas, backtracking, BFS, DFS, A*

2.1. Recursividad.

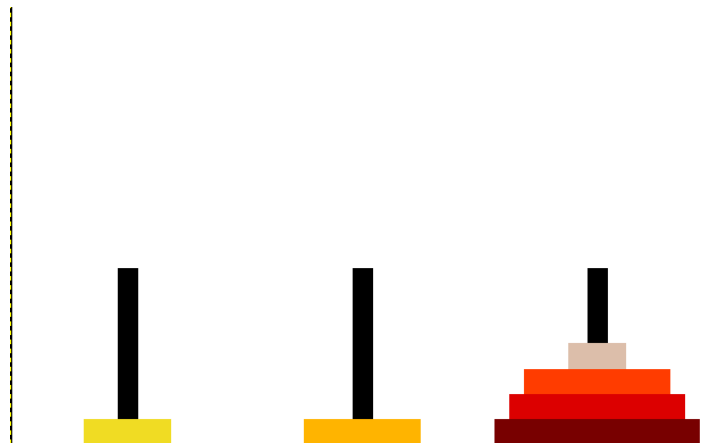
Las recursividad permiten resolver problemas relativamente complejos de forma sencilla. Se basa en una función que se llama así misma, pero con un subproblema del inicial.

Dentro de una función recursiva se tienen dos elementos básicos:

- Caso base: Cuando se cumple finalizan las llamadas y provoca que se vayan resolviendo las llamadas previas en el orden inverso (desapilar).
- Caso recursivo: Se puede tener más de uno en la función, es una llamada a la misma función dentro de esta, con la modificación de algunos parámetros. La función que llama se queda esperando a que la función llamada termina y devuelve un valor si así se ha definido.

En caso de no definir el caso base, o que no se llegue nunca a ejecutar se produce un “stackoverflow” o desbordamiento de la pila (se queda sin memoria en la que apilar las llamadas).

El ejemplo clásico son las Torre de Hanoi:



Fuente:

https://es.wikipedia.org/wiki/Torres_de_Hanoi#/media/Archivo:Iterative_algorithm_solving_a_6_disks_Tower_of_Hanoi.gif

Licencia: Creative Commons. Autor: [Trixx](#)

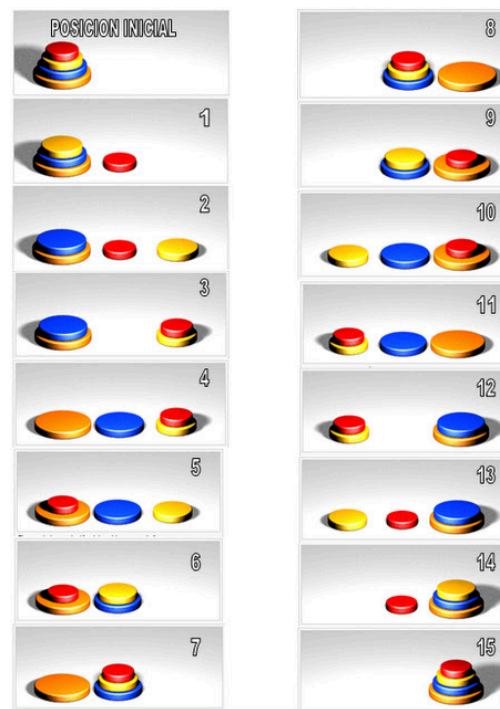
Las reglas son:

- Solo se puede mover un disco cada vez y para mover otro los demás tienen que estar en postes.
- Un disco de mayor tamaño no puede estar sobre uno más pequeño que él mismo.
- Solo se puede desplazar el disco que se encuentre arriba en cada poste.

La resolución de este problema sin usar la recursividad es compleja, en cambio usando la recursividad, en Java:

```
void moverDiscos (int n, int origen, int destino, int aux){
    if (n > 0){
        moverDiscos (n-1, origen, aux, destino);
        printf("Muevo un disco de %d a %d\n", origen, destino);
        moverDiscos (n-1, aux, destino, origen);
    }
}
```

Un ejemplo de ejecución con 3 discos:



Fuente:

https://commons.wikimedia.org/wiki/File:Tower_of_Hanoi_TORRE_DE_LUCAS_TORRE_BRAHMA_solucion.jpg

Licencia: Creative Commons. Autor: [Mastermorfeu](#)

Otro ejemplo clásico es el cálculo del factorial.

```
def factorial (n):
    if(n<=0):
        raise "El número ha de ser mayor o igual a 1"
    #caso base
    if(n==1):
        return 1
    #caso recursivo
    else:
        return n*factorial(n-1)
print (factorial(3))
```

Destacar que pueden existir varios caso recursivos o varias llamadas recursivas en una función y varios casos base (menos común), el ejemplo típico es el cálculo de la función de Fibonacci:

$$f_n = f_{n-1} + f_{n-2}$$

```
def fibonacci(n):
```

```

if(n<0):
    raise "El número ha de ser mayor o igual a 1"
#dos casos base, aunque podría ser solo 1 n<=1
if(n==0):
    return 0
if(n==1):
    return 1
#dos llamadas recursivas a la misma función
return fibonacci(n-1)+fibonacci(n-2)

```

2.2. Estado y espacio de búsqueda.

Cuando se está resolviendo un problema, encontrar la solución puede ser encontrar uno o varios estados que cumplan con las condiciones de la solución, por ejemplo, el clásico juego de las 3 en raya, para garantizar que se gana, se pueden calcular todos los posibles movimientos a partir de un estado del tablero y seleccionar aquel que finalice con una victoria, independientemente de los movimientos que realice el contrario (existen algoritmos específicos para este tipo de problemas, que no se tratan en el curso, como en Mix-Max, o el Alpha-Beta). A cada posible tablero se le denomina estado, que posee el turno del jugador actual y como se encuentra el tablero en ese instante (dependerá del problema), siendo el conjunto de todos los estados posibles es espacio de búsqueda.

Para problemas pequeños, como el anterior, calcular todos los posibles estados no supone un cálculo computacional problemático, en el caso de problemas mayores, se hace inmanejable computacionalmente.

Unas definiciones más formales:

Estado: Un estado es una configuración concreta del problema en un momento dado. Representa una situación posible del sistema.

Espacio de búsqueda: Conjunto de todos los estados posibles junto con las transiciones (acciones o reglas) que permiten pasar de un estado a otro. Es el “mapa completo” de todas las posibilidades del problema. Se puede representar como un árbol (si cada transición expande un estado nuevo) o como un grafo (si puede haber ciclos o varios caminos hacia un mismo estado). Incluye:

- Estado inicial.
- Conjunto de acciones válidas.
- Estados sucesores.
- Posible(s) estado(s) objetivo.

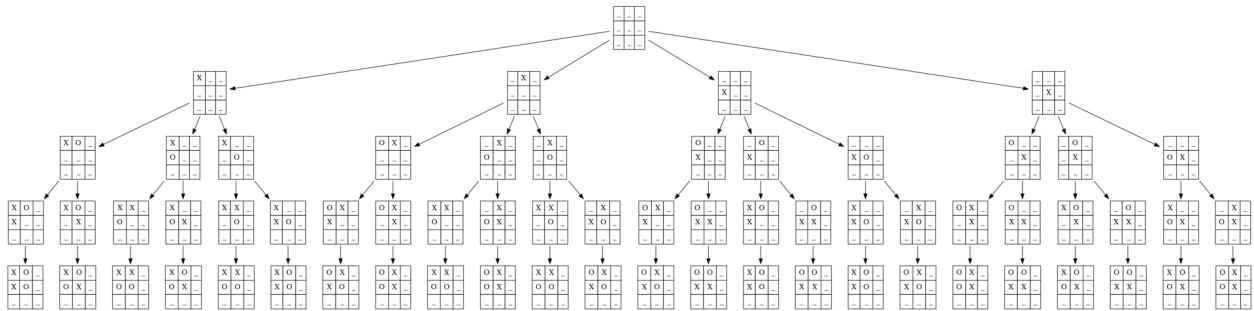
Continuando con el juego de las 3 en raya, existen 3^9 posibles tableros diferentes, es decir 19683, un espacio de estados no muy grande computacionalmente, pero significativo, en calcular todos los posibles estados se tarda unos 11 segundos.

- El estado inicial es una matriz de 3x3 vacía.
- Los conjuntos de acciones válidas son simplemente asignar un nuevo valor ['X','O'].
- Estados sucesores son los nodos que poseen un tablero con la nueva marca.
- Posibles estados objetivos. Nodos con tableros llenos.

Una posible implementación de la creación de todos los posibles estados de forma recursiva:

```
import random
from enum import Enum
from dataclasses import dataclass, field
from typing import List
import itertools
import copy
EMPTY='_'
#enum de Player
class Player(Enum):
    Player1= 'X'
    Player2= 'O'
@dataclass
class Node:
    board:List[List[str]]
    turn:Player
    childrens: List["Node"]= field(default_factory=list)
#cierto si el board está lleno
def full(board):
    for f in board:
        for c in f:
            if c==EMPTY:
                return False
    return True
#genera todos los tableros, cada uno de ellos es un estado, y el conjunto es el espacio de estados
def create_boards(board,turn,level)->Node:
    #se crea el nodo para añadirlo al árbol
    node =Node(board[:],turn)
    if(not full(board) and level>=0):
        for i in range (len(board)):
            for j in range (len(board[i])):
                #se coloca X o O si está libre
                if(board[i][j]==EMPTY):
                    copy_of_board = copy.deepcopy(board)
                    copy_of_board[i][j]=turn
                    #se llama a la recursión con el nuevo tablero
                    next_turn = Player.Player1 if turn == Player.Player2 else Player.Player2
                    children=create_boards(copy_of_board,next_turn,level-1)
                    if(children!=None):
                        node.childrens.append(children)
        return node
    else:
        return None
init_player = random.choice(list(Player))
board= [
    [EMPTY, EMPTY, EMPTY],
    [EMPTY, EMPTY, EMPTY],
    [EMPTY, EMPTY, EMPTY]
]
#tablero vacio, jugador inicial y nivel hasta donde llega
tree=create_boards(board,init_player,4)
```

Un ejemplo, limitado en anchura y profundidad dado el tamaño del gráfico, en la que solo se da valor a unos poco hijos hasta profundidad 4:

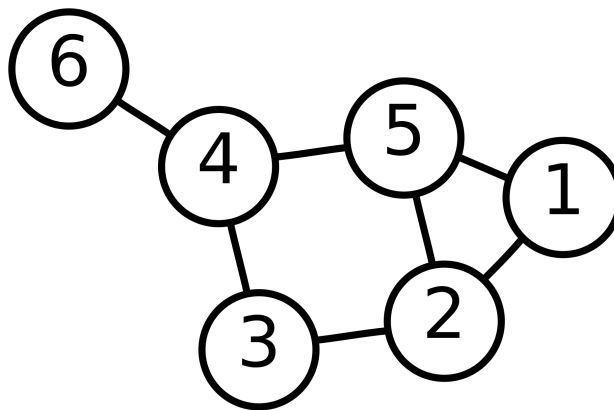


2.3. Definición y representación básica de grafos.

Conviene recordar la definición de grafo en el ámbito de la programación, ya que muchos algoritmos se basan en recorrerlos:

Grafo básico (G) se define como un par $G=(V,E)$:

- V: conjunto de vértices (nodos).
- E: conjunto de aristas, que son pares de vértices.



Representación gráfica de un grafo. Fuente: <https://es.wikipedia.org/wiki/Grafo#/media/Archivo:6n-graf.svg>

Licencia: Dominio Público. Autor: AzaToth.

La representación matemática del grafo anterior:

$$G=\{1,2,3,4,5,6\},\{\{1,2\},\{1,5\},\{2,2\},\{2,3\},\{2,5\},\{3,4\},\{3,5\},\{4,5\},\{4,6\}\}$$

Algunos de los tipos de grafos existentes:

- No dirigido: las aristas no tienen dirección.
- Dirigido (dígrafo): las aristas tienen una dirección (flecha).
- Ponderado: las aristas tienen un peso o coste asociado (distancia, tiempo, capacidad, etc.).
- No ponderado: todas las aristas se consideran iguales.

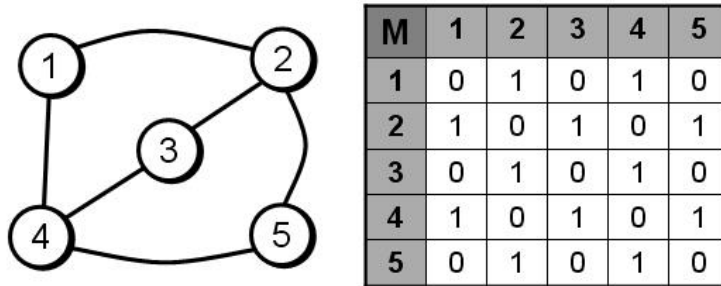
En caso de que las aristas tengan pesos, se añade a la definición el conjunto de pesos:

Grafo ponderado (G) se define como un par $G=(V,E,w)$:

- V: conjunto de vértices (nodos).
- E: conjunto de aristas, que son pares de vértices.
- w: Conjunto de pesos, cuyo índice son aristas, por ejemplo $w(A,B)=5$, indica que la arista entre los vértices A,B es 5..

Un grafo se puede representar de diferentes formas usando estructuras de datos, las dos principales:

Matriz de adyacencia: matriz $n \times n$ donde $mat[i][j] = 1$ (o peso) si existe arista entre los nodos i y j.

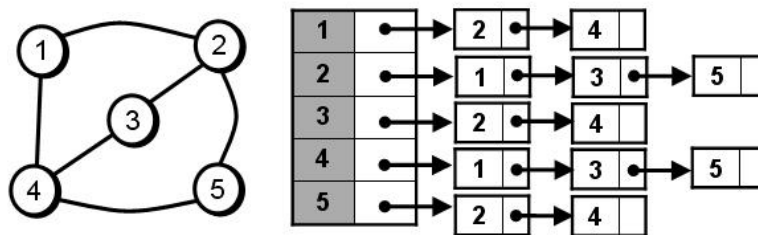


Representación del grafo usando matriz de adyacencia.

Fuente: https://es.wikipedia.org/wiki/Grafo#/media/Archivo:Matriz_de_adyacencia.jpg

Licencia: Dominio Público. Autor: Hebep

Lista de adyacencia: cada nodo almacena la lista de nodos a los que está conectado.



Representación del grafo usando lista de adyacencia. Fuente:

https://es.wikipedia.org/wiki/Grafo#/media/Archivo:Listas_de_adyacencia.jpg

Licencia: Dominio Público. Hebep

Dentro de los grafos, otro concepto destacado es el de camino, que se define como:

Un camino P de longitud k es una secuencia de vértices:

$$v_0, v_1, v_2, \dots, v_k$$

tal que para cada i con $0 \leq i < k$, existe una arista $(v_i, v_{i+1}) \in E$

En los caminos existen algunos con unas características destacadas:

- Camino Hamiltoniano: Un camino hamiltoniano en un grafo es un camino que visita cada vértice exactamente una vez.
- Camino Euleriano: Un camino euleriano en un grafo es un camino que recorre cada arista exactamente una vez.

2.4. Búsqueda en profundidad (DFS).

Técnica de búsqueda no informada, se puede aplicar a grafos definidos previamente o a árboles que se van generando a medida que el algoritmo realiza la búsqueda (como en la búsqueda recursiva vista en el punto anterior).

Algunos de los ámbitos en que se utiliza:

Ámbito / Campo	Tipo de problema que resuelve	Ejemplo concreto
Inteligencia Artificial y Juegos	Búsqueda en espacios de estados, exploración de todas las jugadas posibles	Árbol de jugadas en ajedrez o 3 en raya
Redes y Sistemas	Detección de conexiones, ciclos y componentes	Ver si una red de computadoras está totalmente conectada
Optimización / Rutas	Búsqueda de caminos posibles entre dos puntos	Encontrar todos los caminos posibles en un mapa o grafo de transporte
Bases de datos / Estructuras	Recorridos jerárquicos, árboles, grafos	Recorrer jerarquías de carpetas o estructuras XML/JSON
Compiladores	Análisis de dependencias y sintaxis	Recorrer árboles de sintaxis abstracta (AST)
Ciberseguridad	Detección de vulnerabilidades en grafos de permisos o redes	Encontrar rutas de acceso no autorizadas
Procesamiento de imágenes / grafos	Segmentación en grafos, detección de regiones conexas	Identificar regiones conectadas en una imagen binaria
Web / Crawlers	Recorrer grafos de enlaces	Exploración profunda de páginas web enlazadas

Existe versión recursiva:

```
DFS(Grafo G, Nodo inicio):  
    marcar inicio como visitado  
    procesar(inicio)    // acción deseada (ej: imprimir)  
  
    para cada vecino en G.adj[inicio]:  
        si vecino no está visitado:  
            DFS(G, vecino)
```

Y versión basada en pila:

```
DFS_Iterativo(Grafo G, Nodo inicio):  
    crear una pila vacía  
    crear un conjunto/marca de visitados
```

```

apilar(inicio)
mientras pila no esté vacía:
    nodo ← desapilar()

    si nodo no está en visitados:
        marcar nodo como visitado
        procesar(nodo) // acción deseada (ej: imprimir)

    para cada vecino en G.adj[nodo]:
        si vecino no está visitado:
            apilar(vecino)

```

En el caso de problemas con exploración de estados:

Recursivo:

```

DFS_Recursivo(estado):
    si es_final(estado):
        procesar_final(estado)
        retornar
    procesar(estado) // opcional
    para cada sucesor en generar_vecinos(estado):
        DFS_Recursivo(sucesor)

```

Iterativo basado en pila:

```

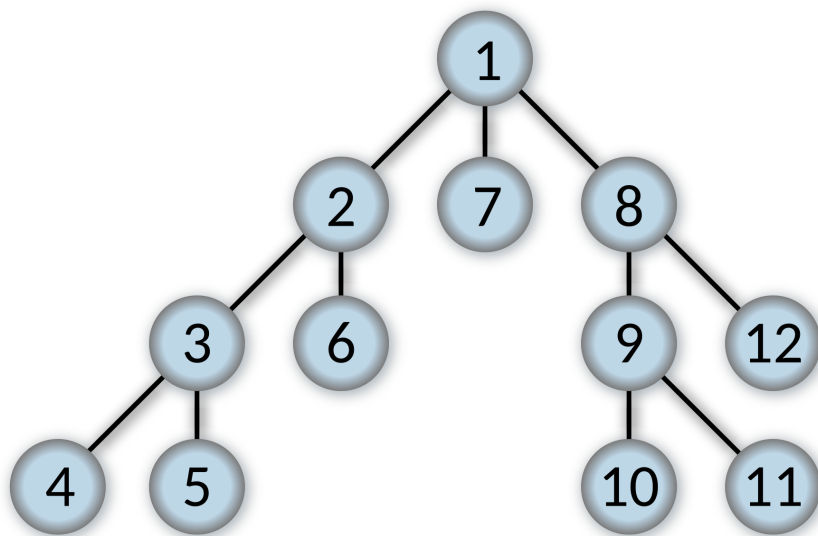
DFS_Pila(estado_inicial):
    crear pila vacía
    apilar(estado_inicial)

    mientras pila no esté vacía:
        estado ← desapilar()
        si es_final(estado):
            procesar_final(estado)
            continuar

        para cada sucesor en generar_vecinos(estado):
            apilar(sucesor)

```

En la siguiente imagen se puede ver el recorrido utilizando DFS, se puede observar cómo hasta que no alcanza la profundidad máxima, no vuelve hacia arriba en el árbol.



Recorrido DFS

Fuente: https://es.wikipedia.org/wiki/B%C3%BAsqueda_en_profundidad#/media/Archivo:Depth-first-tree.svg.

Licencia: Creative Commons. Autor: Alexander Drichel

2.5. Búsqueda en anchura (BFS).

Otras de las técnicas no informadas, que también se puede aplicar a grafos o a exploración de estados.

Ámbito / Campo	Tipo de problema que resuelve	Ejemplo concreto
Informática / Teoría de grafos	Búsqueda del camino más corto en grafos no ponderados	Encontrar el camino más corto entre dos nodos en una red de computadoras
Inteligencia Artificial	Exploración sistemática de espacios de estados	Resolver un rompecabezas como el "8-puzzle" utilizando búsqueda por niveles
Redes de comunicación	Determinación de rutas mínimas en redes	Encontrar la ruta más corta para enviar un paquete en una red de routers
Robótica / Navegación	Planificación de rutas	Navegar un robot desde un punto inicial a un destino evitando obstáculos
Análisis de redes sociales	Análisis de relaciones y conexiones	Encontrar la distancia social entre dos personas en una red social (grados de amistad)

Web Crawling	Exploración estructurada de sitios web	Un crawler que explora sitios web a partir de una URL raíz y sigue enlaces por nivel
Videojuegos (IA de enemigos)	Movimiento de personajes en mapas por niveles	Un enemigo que busca al jugador en un mapa dividido por celdas
Biología computacional	Análisis de relaciones genéticas o moleculares	Encontrar relaciones mínimas entre genes en una red genética
Seguridad informática	Detección de vulnerabilidades en redes	Explorar todos los dispositivos conectados en una red para detectar vulnerabilidades

Al igual que DFS, este algoritmo posee también versiones recursivas e iterativas, aunque se suele utilizar la versión iterativa ya que la recursiva entre otros problemas hace un uso ineficiente de la pila de llamadas, no mejora el rendimiento y su implementación no es muy natural.

Versión iterativa:

```

BFS_Cola(estado_inicial):
    crear cola vacía
    encolar(estado_inicial)

    mientras cola no esté vacía:
        estado ← desencolar()
        si es_final(estado):
            procesar_final(estado)
            continuar

        procesar(estado)    // opcional

        para cada sucesor en generar_vecinos(estado):
            encolar(sucesor)

```

2.6. Backtracking.

Este algoritmo se basa en no explorar nodos que se sabe que no van a llevar un estado aceptable, volviendo atrás en la búsqueda. Se puede hacer el símil en la vida real con moverse por una ciudad antigua en forma de laberinto: se prueba un camino, y si te lleva a un callejón sin salida, se retrocede y se prueba otro camino. Se basa en:

- **Búsqueda en árbol:** El espacio de soluciones se representa como un árbol de decisiones, donde cada nodo representa una elección parcial.
- **Construcción incremental de soluciones:** Se construye una solución paso a paso (recursivamente o iterativamente), de forma similar a los algoritmos vistos anteriormente.
- **Podas:** Si en algún punto una solución parcial no puede llevar a una solución completa válida, se descarta (corta/poda) ese camino y se vuelve atrás.

- **Recursión + Deshacer (backtrack):** Después de probar una opción, si no sirve, se deshace la elección y se prueba otra.

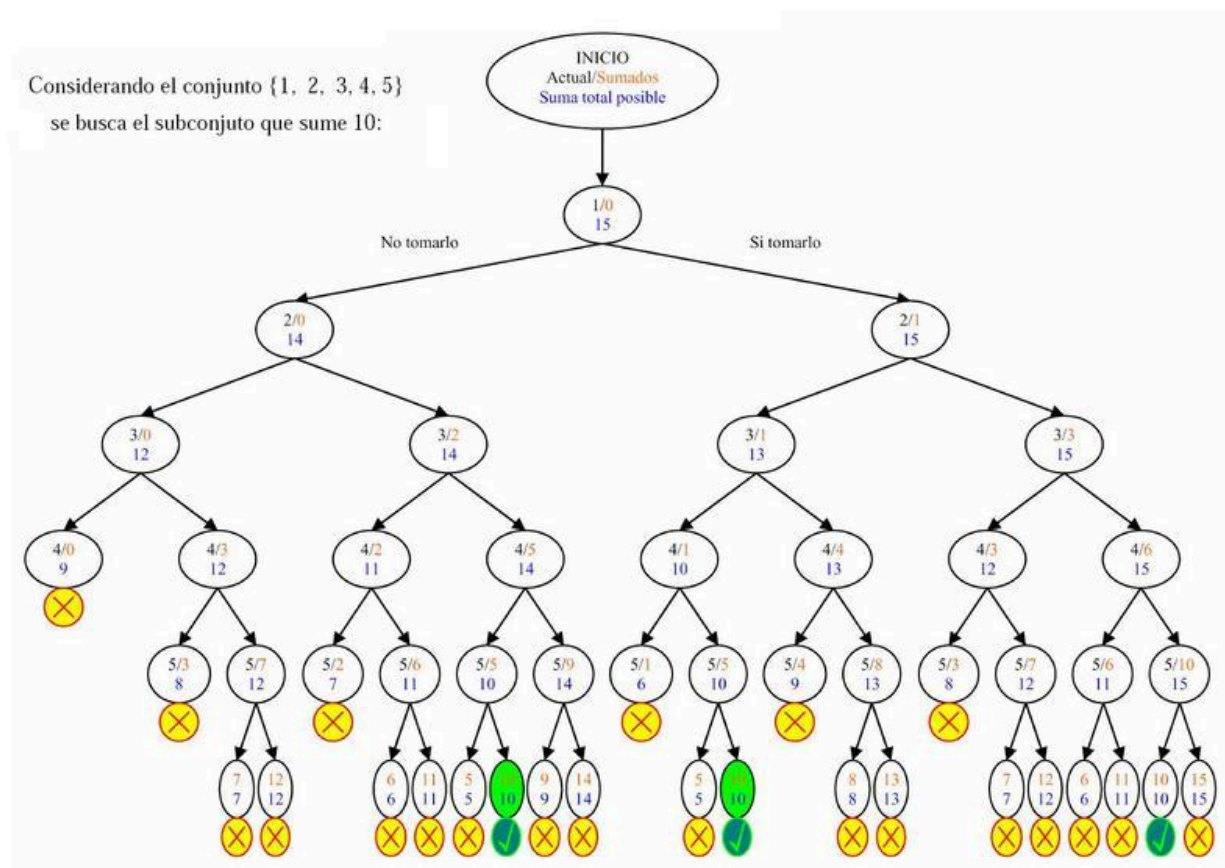
Los problemas que resuelve este algoritmo poseen las siguientes características:

- Hay múltiples opciones a considerar.
- La solución se puede construir paso a paso.
- Se puede verificar si una solución parcial es válida.

Y se centra en:

- Toma de decisiones: Búsqueda de las soluciones que satisfacen ciertas restricciones.
- Optimización : Búsqueda de la mejor solución en base a una función objetivo.

Un ejemplo extraído de la Wikipedia, en la que se busca en un subconjunto de números que sumen una cantidad concreta a partir de conjunto de números, en el ejemplo 10, y el conjunto es {1, 2, 3, 4, 5}:



Árbol de recursión de algoritmo Backtracking para calcular subconjunto que sume n, en un conjunto

Fuente: https://commons.wikimedia.org/wiki/File:Branch%26bound_low.jpg

. Licencia: Dominio público. Autor: [Moctel3](#)

En cada iteración del algoritmo se decide entre explorar o no el siguiente elemento/estado del espacio de estados, en el problema anterior la primera decisión es si usar o no usarlo, para el caso de usarlo, sus hijos son usarlo o no usarlo... así se recorre todo el espacio de estados. Existen situaciones en las que no es necesario seguir explorando:

1. La suma actual más los elementos pendientes de insertar son menores a n (no se ha tenido en cuenta en la imagen).
2. La suma actual no es n, y sumar el mínimo de los pendientes por usar es mayor de n.

En estos casos se para la exploración y se devuelve al padre que no esa rama no conlleva a ninguna solución, válida (vuelta atrás), explorando el padre el siguiente estado en busca de una solución.

En la imagen cuando se tiene el subconjunto {1,2,3,4} no tiene sentido explorar el {5} ya que la suma es mayor de 10 (caso 2).

Otro ejemplo, representando en un vector (si se toma o no el número con 0/1 dependiendo de la posición, {0,1,1,x,x}, indica que no se ha tomado el 1, se ha tomado el 2 y 3 y quedan por evaluar el 4 y 5). Si se encuentra en un estado {0,1,1,1,x}, da igual el valor de x {0,1}, ya que sino se toma la suma es 9, y si se toma, la suma es 14, no es necesario explorarlo, pasando al siguiente caso {0,1,1,0,x}.

Algunos de los usos más comunes:

Problema	Cómo se aplica el Backtracking
Sudoku	Llenar una celda, verificar restricciones, retroceder si hay conflicto
N-reinas	Colocar una reina por fila, retroceder si hay amenaza
Coloreo de grafos	Asignar colores a nodos sin repetir en adyacentes
Laberintos	Avanzar paso a paso, retroceder en callejones sin salida
Anagramas / Permutaciones	Generar combinaciones de letras o elementos
Problemas de mochila	Seleccionar subconjuntos que maximizan peso/valor sin pasarse
Puzzles (ej. 8-puzzle)	Probar movimientos secuenciales hasta resolver el puzzle

Los problemas se definen:

Conjunto de variables:

$$X=\{x_1, x_2, \dots, x_n\}$$

Dominio de las variables x_i :

$$D_i=\{v_{i1}, v_{i2}, \dots, v_{im}\}$$

Restricciones que han de cumplir las variables:

$$R=\{R_1, R_2, \dots, R_k\}$$

Estado:

$$A=\{x_1=a_1, x_2=a_2, \dots, x_n=a_n\} \text{ con } a_i \in D_i$$

Y se resuelven cuando se encuentra un estado que cumple:

$$R_j(A) = \text{verdadero} \quad \forall j \in \{1, \dots, k\}$$

donde R_j es una función que devuelve cierto si se cumplen todas las restricciones R .

El pseudocódigo del algoritmo:

```
1. función backtracking(solución_parcial) → Solución o NULO
2.     si solución_parcial es completa entonces
3.         devolver copia_de(solución_parcial)
4.     fin si
5.
6.     para cada opción en opciones_posibles(solución_parcial) hacer
7.         si es_valida(opción, solución_parcial) entonces
8.             agregar opción a solución_parcial
9.             resultado ← backtracking(solución_parcial)
10.            si resultado ≠ NULO entonces
11.                devolver resultado      // se encontró una solución
12.            fin si
13.
14.            eliminar opción de solución_parcial
15.        fin si
16.    fin para
17.
18.    devolver NULO    // ninguna opción produjo solución
19. fin función
```

En la línea 2 se tiene el caso base de cualquier algoritmo recursivo, en este caso que la solución parcial cumpla con todas las restricciones.

Línea 6, para cada estado se calculan los siguientes estados.

Línea 7, se evalúa si el nuevo estado calculado en 6 es válido (puede llegar a ser una solución).

Línea 8, se hace una copia de la solución parcial con la nueva opción.

Línea 9, se llama a la recursión, si devuelve un estado, indica que este estado es válido.

Línea 14, en caso de no haber encontrado una solución válida en ese estado, se prueba con el siguiente estado, vuelve a 7 con una nueva combinación..

Un factor determinante es la selección del siguiente estado, existiendo diferentes estrategias:

Estrategia	Descripción	Pros	Contras
Orden natural (secuencial)	Explora las opciones en orden fijo y sistemático (por ejemplo, de izquierda a derecha o de 1 a N).	- Fácil de implementar. - Garantiza explorar todo el espacio de búsqueda.	- Muy ineficiente si hay muchas combinaciones. - No prioriza opciones prometedoras.

Filtrado previo (opciones válidas)	Antes de intentar una opción, se descartan las que violan restricciones conocidas.	- Reduce el número de ramas. - Mejora el rendimiento sin cambiar la lógica básica.	- Requiere funciones de validación eficientes. - No cambia el orden de exploración.
MRV (Minimum Remaining Values)	Elige la variable o posición con menos opciones válidas (la más restringida).	- Detecta callejones sin salida rápidamente. - Reduce retrocesos innecesarios.	- Aumenta el costo de cálculo en cada paso. - Solo útil si hay múltiples variables.
Valor menos restrictivo (Least Constraining Value)	Entre las opciones válidas, elige la que menos restringe las decisiones futuras.	- Favorece soluciones viables. - Complementa bien a MRV.	- Requiere evaluar el impacto futuro de cada opción. - Coste computacional adicional.
Aleatorio (Randomized Backtracking)	Selecciona opciones en orden aleatorio o con reinicios aleatorios.	- Puede encontrar soluciones distintas en cada ejecución. - Útil en entornos con múltiples soluciones.	- No garantiza reproducibilidad. - Puede tardar más si el azar no favorece buenas ramas.
Poda (Branch and Bound)	Descarta ramas que no pueden mejorar una solución parcial (según un límite o heurística).	- Reduce drásticamente el espacio de búsqueda. - Ideal para optimización (mochila, viajante).	- Requiere función de costo y límite bien definidos. - Complejidad adicional en la implementación.
Heurística de orden adaptativo	Cambia el orden de exploración dinámicamente según el estado actual.	- Se adapta a la evolución del problema. - Puede combinarse con otras estrategias.	- Implementación compleja. - Puede requerir información global del problema.

En las dos últimas estrategias se utilizan heurísticas (técnica de la indagación y del descubrimiento, usando el conocimiento actual y realizando un cálculo del futuro), con lo que pasa a ser un algoritmo informado. En este tema solo se trata la selección por orden natural.

Un ejemplo de implementación del algoritmo de Backtracking en Python, donde se tiene el problema de obtener n , sumando elementos de un conjunto de números, el problema definido matemáticamente:

Conjunto de variables:

$X=\{x_1, x_2, \dots, x_n\}$, vector que indica si el número x_i es o no parte de la solución

Dominio de las variables x_i :

$D_i = \{0,1\}$, 0 el número en la posición i no forma parte de la solución

Restricciones que han de cumplir las variables:

$$R = (\sum(R_i) = n)$$

Solución:

$$R_j(A) = \text{verdadero} \quad \forall j \in \{1, \dots, k\}$$

Una de las claves del algoritmo es generar los estados posibles, en el siguiente código se generan todas las posibles soluciones (válidas o no):

```
def pre_backtracking(elementos:list[int],
solucion_parcial:list[bool],objetivo:int)->list[bool]:
    print (solucion_parcial)
    #caso base para parar la búsqueda
    if (len(solucion_parcial)==len(elementos)):
        return None
    """
    se define el dominio, en este caso todas las variables tienen el mismo
    se puede definir también a nivel global
    """
    dominio=[True,False]
    #se ha llegado al final y esta no es solucion
    for opcion in [0,1]:
        siguiente_estado=siguiente_solucion(solucion_parcial,opcion)
        pre_backtracking(elementos,siguiente_estado,objetivo)
    return None
```

Si se ejecuta, se puede ver cómo se generan todas las posibilidades:

```

PS C:\Users\user\Desktop\mia> python .\backtraking.py
[]
[0]
[0, 0]
[0, 0, 0]
[0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 1]
[0, 0, 0, 1]
[0, 0, 0, 1, 0]
[0, 0, 0, 1, 1]
[0, 0, 1]
[0, 0, 1, 0]
[0, 0, 1, 0, 0]
[0, 0, 1, 0, 1]
[0, 0, 1, 1]
[0, 0, 1, 1, 0]
[0, 0, 1, 1, 1]
[0, 1]
[0, 1, 0]
[0, 1, 0, 0]
[0, 1, 0, 0, 0]
[0, 1, 0, 0, 1]
[0, 1, 0, 1]
[0, 1, 0, 1, 0]
[0, 1, 0, 1, 1]
[0, 1, 1]
[0, 1, 1, 0]
[0, 1, 1, 0, 0]
[0, 1, 1, 0, 1]
[0, 1, 1, 1]
[0, 1, 1, 1, 0]
[0, 1, 1, 1, 1]
[1]

```

Ahora queda implementar las funciones:

- `es_solucion(elementos: list[int], solucion_parcial: list[bool], objetivo: int)-> bool`
- `siguiente_solucion(solucion_parcial:list[bool],siguiente:bool)->list[bool]`
- `es_valido(elementos:list[int], solucion_parcial:list[bool],objetivo:int)->bool`

Y al integrarlas en el algoritmo:

```

def backtraking(elementos:list[int], solucion_parcial:list[bool],objetivo:int)->list[bool]:
    if(es_solucion(elementos,solucion_parcial,objetivo)):
        return solucion_parcial
    #se ha llegado al final y esta no es solucion
    elif (len(solucion_parcial)==len(elementos)):
        return None
    else:
        for opcion in [True, False]:
            siguiente_estado=siguiente_solucion(solucion_parcial,opcion)
            #si es valido se devuelve, sino se calcula el siguiente
            if(es_valido(elementos,siguiente_estado,objetivo)):
                calculado=backtraking(elementos,siguiente_estado,objetivo)
                if (calculado!=None):
                    return calculado
        #no se ha encontrado solucion
        return None

```

2.7. A*.

Los **algoritmos de búsqueda informada** utilizan información sobre el camino recorrido y pueden apoyarse en una estimación de cuál es el mejor camino pendiente por explorar. Esta estimación se llama **heurística** y se denota como $h(n)$.

El algoritmo más conocido de este tipo es **A***, cuyo objetivo es encontrar el **camino de menor coste** desde un nodo inicial hasta el objetivo. A* funciona con una **cola ordenada** para los nodos pendientes de visitar y un set para los nodos visitados que determina el siguiente nodo a explorar y una, seleccionando aquel que minimice la expresión:

$$f(n)=g(n)+h(n)$$

donde:

- $g(n)$ es el coste real desde el nodo raíz hasta el nodo n .
- $h(n)$ es la estimación del coste desde n hasta el objetivo.

Para que $h(n)$ sea adecuada, se compara con $h^*(n)$, que representa el **coste real mínimo restante** desde el nodo n hasta la meta. Una heurística $h(n)$ se considera **admisible** si cumple:

$$h(n) \leq h^*(n) \quad \forall n$$

Es decir, la estimación nunca debe exceder el coste real restante. Si se cumple esta condición, A* garantiza encontrar un **camino de coste mínimo**.

En cambio, si en **algún nodo se cumple**:

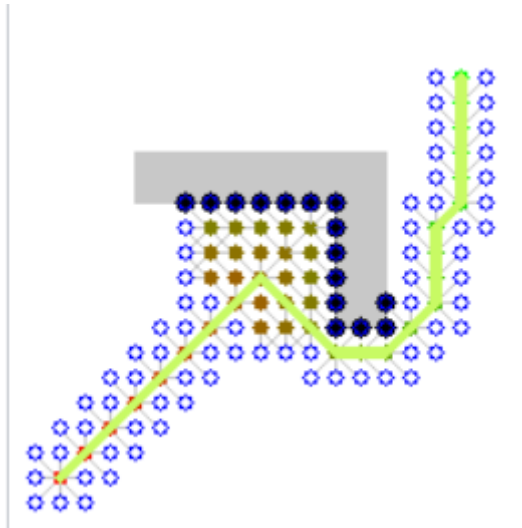
$$h(n) > h^*(n)$$

se dice que $h(n)$, o $h^*(n)$, es una **heurística sobreestimada**, lo que implica que:

- Algunos caminos pueden parecer más caros de lo que realmente son, y
- A* **podría no elegir la solución óptima**, aunque encontrará alguna solución válida.

Tipo	Condición	Consecuencia
Admisible	$h(n) \leq h^*(n)$ (nunca sobreestima el coste real)	A* encuentra la solución óptima
Sobreestimada	($h(n) > h^*(n)$) en algún nodo	A* puede ignorar caminos óptimos , encontrando soluciones subóptimas

Un ejemplo de exploración de un mapa, para llegar de un punto a otro con obstáculos:



Mapa aplicación algoritmo A* para llegar de un punto a otro

Fuente: https://en.wikipedia.org/wiki/A*_search_algorithm#/media/File:Weighted_A_star_with_eps_5.gif

Licencia: Creative Commons. Autor: [Subh83](#)

Algunos de los usos de A* son:

Área	Ejemplo de uso	Descripción
Navegación y rutas	GPS, robots móviles, drones	Encuentra la ruta más corta entre dos puntos considerando distancias, obstáculos, o costes.
Videojuegos (IA de enemigos o NPCs)	Pathfinding de personajes	Los NPCs usan A* para moverse de forma natural evitando muros y trampas.
Planificación en IA	Planificación de acciones	Planifica secuencias de acciones con costes para alcanzar un objetivo.
Robótica y logística	Vehículos autónomos, almacenes automáticos	Calcula rutas óptimas para robots en entornos con múltiples obstáculos.
Redes y telecomunicaciones	Enrutamiento de datos	Encuentra rutas óptimas entre nodos en una red, minimizando latencia o coste.
Juegos de tablero y resolución de puzzles	8-puzzle, 15-puzzle, cubo de Rubik	Busca la secuencia mínima de movimientos para alcanzar la solución.
Bioinformática y optimización	Alineamiento de secuencias, ensamblaje de genomas	Encuentra caminos óptimos en grafos de datos biológicos.

Diseño asistido por computadora (CAD)	Rutas de cableado, impresión 3D	Encuentra trayectorias eficientes en diseños complejos.
--	---------------------------------	---

El pseudocódigo del algoritmo:

```

Función A_Star(inicio, objetivo):
    // Inicializar estructuras
    abierto = cola_prioridad()           // nodos por explorar
    cerrado = conjunto()                 // nodos ya explorados

    g[inicio] = 0                        // coste desde inicio hasta el nodo
    h[inicio] = heurística(inicio)       // estimación hasta objetivo
    f[inicio] = g[inicio] + h[inicio]

    abierto.insertar(inicio, f[inicio])
    Mientras abierto no esté vacío:
        nodo_actual = abierto.extraer_min() // nodo con f(n) más bajo

        Si nodo_actual == objetivo:
            retornar reconstruir_camino(nodo_actual)
        cerrado.agregar(nodo_actual)
        Para cada vecino de nodo_actual:
            Si vecino en cerrado:
                continuar

            coste_temporal = g[nodo_actual] + coste(nodo_actual, vecino)

            Si vecino no está en abierto o coste_temporal < g[vecino]:
                g[vecino] = coste_temporal
                h[vecino] = heurística(vecino)
                f[vecino] = g[vecino] + h[vecino]
                vecino.padre = nodo_actual

            Si vecino no está en abierto:
                abierto.insertar(vecino, f[vecino])

    Retornar "No se encontró camino"

```

La elección de la **heurística h(n)** en A* es **crítica**, porque determina cómo el algoritmo estima el coste restante hasta el objetivo y, por tanto, influye en la eficiencia y en la optimalidad del camino encontrado. Algunas de las heurísticas más conocidas en función del tipo de problema:

Tipo de problema	Heurística	Expresión matemática
Cuadrícula ortogonal (mov. H/V)	Distancia Manhattan	La distancia euclídea en 2 dimensiones se calcula: $d_E(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
Cuadrícula con diagonales	Distancia Euclidiana	Distancia de Manhattan o métrica del taxista de dos vectores p,q en un espacio vectorial de n-dimensiones con un sistema de coordenadas cartesianas como:

		$d_1(\mathbf{p}, \mathbf{q}) = \ \mathbf{p} - \mathbf{q}\ _1 = \sum_{i=1}^n p_i - q_i ,$
Cuadrícula con diagonales, coste igual	Distancia Chebyshev	La distancia de Chebyshev entre dos vectores o puntos p y q , de coordenadas normales p_i y q_i , respectivamente, es: $D_{\text{Chebyshev}}(p, q) := \max_i (p_i - q_i).$
Laberintos complejos o grafos generales	Coste mínimo de arista conocida	$d_{\min} = \min\{\text{costo}(a, b) \mid (a, b) \in E\}$
8-puzzle / N-puzzle	Piezas fuera de lugar	$h(n) = \sum_{i=1}^N [p_i \neq p_i^{\text{goal}}]$
8-puzzle / N-puzzle	Distancia Manhattan	
Problemas de rutas en mapas	Distancia Euclidiana	
Problemas de rutas con carreteras	Distancia geodésica / Haversine	
Robótica o planificación de movimiento	Distancia ponderada Euclidiana	$h(n) = w \times \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$

Ejemplo con pymaze

```

from pymaze import maze, agent, COLOR, textLabel
from pymaze.search import AStar

# Crear un laberinto de 10x10
m = maze(10, 10)
m.CreateMaze(loopPercent=20) # loopPercent agrega algunas rutas extra

# Resolver el laberinto usando A*
solution = AStar(m).search() # Retorna un diccionario con la ruta óptima

# Mostrar la ruta óptima en el laberinto
m.highlightPath(solution, color=COLOR.red) # Resalta el camino en rojo

# Crear un agente para recorrer el laberinto visualmente

```

```

a = agent(m, footprints=True, color=COLOR.blue, algo=AStar)

# Añadir una etiqueta de texto con info
textLabel(m, 'Ruta óptima', f'Pasos: {len(solution)}')

# Iniciar la visualización
m.run()

```

3. Actividades de autoevaluación.

Hoy en día, muchas de las actividades de este módulo pueden resolverse con una simple consulta a una herramienta de inteligencia artificial. Sin embargo, el verdadero valor del aprendizaje no está en obtener la respuesta, sino en **entender el proceso que lleva a ella**.

Cada estudiante tiene en sus manos la decisión: usar la IA como un apoyo para aprender más y mejor, o limitarse a obtener resultados sin comprenderlos. Quienes opten por la segunda opción podrán superar el módulo, pero estarán dejando pasar la oportunidad de adquirir los conocimientos y destrezas que realmente marcan la diferencia en el mundo profesional.

En el sector informático actual, lo que abre puertas no es un título, sino **la capacidad de pensar, resolver problemas y aplicar lo aprendido**.

1. Dibujar el siguiente grafo:

$$V=\{A,B,C,D,E,F\}$$

$$E=\{(A,B),(A,C),(B,D),(C,E),(D,E),(E,F)\}$$

2. Representar el grafo anterior usando listas de adyacencia.

3. Dibujar el siguiente grafo ponderado:

$$V=\{A,B,C,D,E,F\}$$

$$E=\{(A,B),(A,C),(B,C),(B,D),(C,E),(D,E),(D,F),(E,F)\}$$

$$w(A,B)=4, w(A,C)=2, w(B,C)=1, w(B,D)=5, w(C,E)=8, w(D,E)=2, w(D,F)=6, w(E,F)=3$$

4. Representar el grafo anterior usando una matriz de adyacencia.
5. Utilizando las librerías `networkx` (<https://networkx.org/documentation/stable/index.html>) y `matplotlib` dibujar el grafo anterior.
6. Repasando recursividad, implementar los siguientes algoritmos recursivos:
 - Calcular el factorial de un número $n!$, usando recursividad.
 - Implementar una función que de forma recursiva indique si una cadena es palíndromo o no.
 - Sumar una lista de elementos, de forma recursiva.
 - Resolver las torres de Hanoi.

Cada llamada `hanoi(n, A, C, B)`:

- Mueve $n-1$ discos de A a B (usando C como auxiliar).
- Mueve el disco más grande de A a C.

- Mueve los $n-1$ discos de B a C (usando A como auxiliar).
 - Dada una cadena, genera todas las combinaciones posibles de sus letras.
7. Investigar e implementar el recorrido BFS y DFS usando networkx para el grafo del ejercicio 3, desde el nodo A.
8. Implementar las funciones para el algoritmo de Backtracking del ejercicio indicado en los apuntes :
- `es_solucion(elementos: list[int], solucion_parcial: list[bool], objetivo: int) -> bool`
 - `siguiente_solucion(solucion_parcial: list[bool], siguiente: bool) -> list[bool]`
 - `es_valido(elementos: list[int], solucion_parcial: list[bool], objetivo: int) -> bool`

9. El problema de la mochila es el clásico algoritmo que se resuelve entre otros por backtracking (existen otros métodos).
Se define así:

Dado un conjunto de n objetos, cada uno con un peso $w[i]$ y un valor $v[i]$, y una mochila que puede soportar un peso máximo W , determinar qué objetos incluir en la mochila de forma que:

- La suma total de los pesos no exceda la capacidad W .
- La suma total de los pesos sea máxima.

Usar DFS o BFS (a mano, no usar librerías de terceros) y backtracking, comparando tiempos de ejecución para diferentes tamaños del problema.

Existen otras variantes con valores para cada elementos de forma que se maximice o minimice el coste, se opta por hacer la más sencilla, aunque solo sería necesario modificar la evaluación del valor de los resultados calculados. Pasando a ser un problema de optimización.

10. Dado el siguiente mapa, realizar una traza del algoritmo A* indicando el estado de las estructuras de datos.

